McMaster University
Engineering

RoCam

High Performance Vision-Guided Rocket Tracker

Capstone Project
Department of
Computing and Software
Mechatronics &
Software Engineering

Motivation

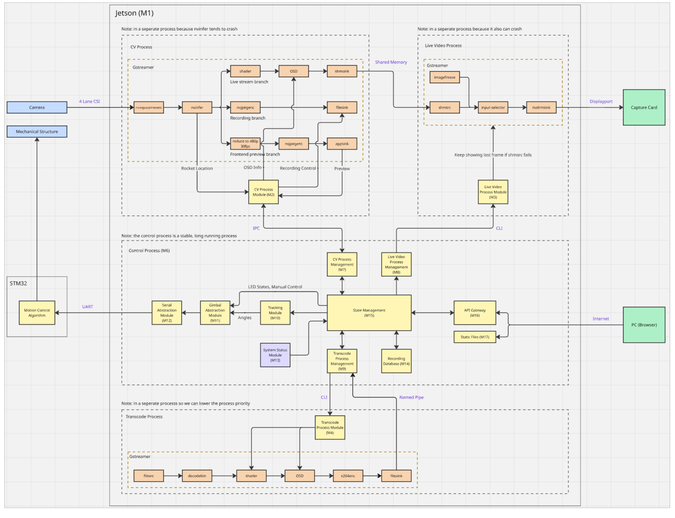
Capturing smooth, stable footage of model rockets is a major challenge. These vehicles accelerate to hundreds of meters per second within fractions of a second, making manual camera tracking nearly impossible. Existing commercial tracking solutions are prohibitively expensive and not designed for amateur rocketry.

RoCam addresses this gap with an **autonomous, vision-guided camera tracking system** that locks onto and follows model rockets in real time. The system delivers smooth **1080p video at 60 fps**, capturing critical flight events such as motor ignition, staging, and parachute deployment.

Designed for small-scale launches (<200 m apogee), the platform is **scalable to high-powered rockets** (3 km+ apogee) for organizations such as the McMaster Rocketry Team.

RoCam Launch and Tracking Operations

System Architecture



Real-Time Vision and Control Pipeline

Edge Compute & Hardware. The implemented architecture follows a distributed multi-process edge-compute design on an NVIDIA Jetson. To ensure high reliability, CPU/GPU-heavy perception workloads are isolated from control and operator services via Inter-Process Communication (IPC). This strict fault isolation preserves deterministic command flow and API stability even if the vision pipeline encounters transient errors during launch-dynamic conditions.

Vision Pipeline. Image acquisition utilizes a GStreamer capture graph configured for 1920×1080 at 60 fps, feeding NVIDIA DeepStream for GPU-accelerated preprocessing. Detection relies on a YOLO network optimized with TensorRT, yielding low-latency normalized bounding boxes. A decoupled Live Video process parallelizes OSD telemetry overlay and HDMI output to guarantee continuous situational awareness without blocking inference.

Kinematic Control. The tracking module calculates image-plane target error (targeting frame center $cx = 0.5, cy = 0.5$) to generate proportional 2-axis gimbal actuation commands. These commands pass through a validated, state-gated pathway—enforcing rate limits and safety bounds—before transmission to an STM32 microcontroller via UART, ensuring deterministic, closed-loop responsiveness throughout boost and coast regimes.

Backend & State Management. System orchestration is governed by a centralized State Machine (Disarmed, Armed, Tracking, Recording) that prevents invalid mode combinations and enforces hardware safety. A Flask-based API Gateway exposes a REST interface for offline-first LAN operation, managing mode transitions, asynchronous recording/transcoding workflows, and streaming SSE telemetry for live state and control-stream observability.

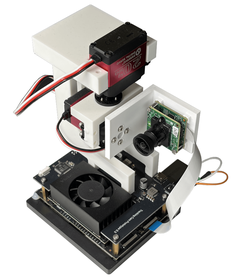
Tools & Technologies

NVIDIA · ST life.augmented · Python · TypeScript · Rust · React · Flask

Software & Hardware Features

Web-Based Operator Interface

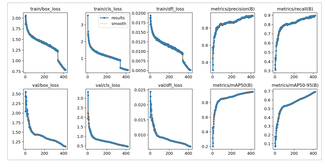
Real-Time Gimbal Control: RoCam provides a low-latency control path between the operator interface and an STM32-based pan–tilt gimbal controller, allowing software commands to be translated into immediate physical camera motion during launch preparation and flight operation. **Manual Positioning:** When the system is in **idle** mode, the operator can use the web interface to manually pan and tilt the camera in real time, making it possible to align the view with the launch rail, expected ascent corridor, or surrounding airspace before ignition. **Remote Operation:** All core control actions are available through a browser-based interface over a local network, so the system can be operated from a safe stand-off distance without requiring physical access to the tracker or any public internet connection. **System-State Awareness:** The interface presents clear visual feedback on the current state of the system, including **idle**, **armed**, and **tracking**, allowing the operator to quickly verify readiness and respond appropriately during time-critical launch procedures. **Safe Override:** At any point, the operator can immediately stop automated motion and command the system to return to a safe **idle** state, which is essential for fault recovery, unexpected target behavior, or general operational safety.



Two-Axis Gimbal Prototype

Autonomous Vision Tracking: Once the system is explicitly placed in the **armed** state, RoCam continuously processes the live camera feed to detect the rocket as a small, fast-moving target under realistic outdoor launch conditions. **Automatic State Transition:** When a valid rocket target is detected, the system automatically transitions from **armed** to **tracking** without requiring further operator input, reducing reaction delay during the most critical phase of flight. **Closed-Loop Target Following:** Detection results from the vision pipeline are converted into continuous pan and tilt corrections so that the gimbal actively follows the rocket and keeps it near the center of the frame throughout ascent and, where possible, descent. **Robustness in Challenging Conditions:** The tracking pipeline is designed to remain effective despite common disturbances such as smoke plume, glare, motion blur, or rapid angular movement, all of which are expected in real launch environments. **Short-Term Re-Acquisition:** If the rocket is briefly lost from view, the system attempts short-term re-acquisition before declaring the target lost, improving overall tracking continuity and increasing the likelihood of preserving useful flight footage without manual intervention.

Computer Vision & Machine Learning



YOLO26s-P2 Training Performance

- **Model Architecture:** The detector was upgraded from the baseline **YOLO26s** to **YOLO26s-P2**, which adds a **stride-4 detection head** for improved sensitivity to very small rockets, at an approximate **+22% GFLOPs** cost while remaining suitable for edge deployment.
- **Data Strategy:** To suppress false positives, the training set was expanded with **4,000 COCO hard-negative background images** with empty labels, increasing the training split from **11,832** to **15,832** images and making background-only samples about **25%** of training data.
- **Training Stages:** Performance initially dropped from **mAP50–95 = 0.694** to **0.614** after introducing the P2 head and harder negatives, then recovered through a **three-stage fine-tuning pipeline**: close-mosaic fine-tuning raised performance to **0.720**, and ultra-fine polishing improved it further to a final **mAP50 = 0.958** and **mAP50–95 = 0.750**.
- **Training Platform:** Experiments were run using **Distributed Data Parallel (DDP)** with mixed precision on a **4× NVIDIA H100 PCIe** cluster, enabling stable large-batch training, reproducible stage-by-stage comparison, and efficient optimization of the final detector.

Our Team

Development Team

Zifan Si
Jianqing Liu
Mike Chen
Xiaotian Lou

Launch Team

McMaster Rocketry Team

Advisory Team

Dr. Shahin Sirouspour
Dr. Spencer Smith
Tanya Djavaherpour